

Foundations of Prompt Engineering & API Access

Session Agenda

- UI vs API - The Hinge Moment
 - Calling LLMs via API
 - Prompt Anatomy
 - Core Prompt Engineering Techniques
 - Live Exercise & Best Practices
-

The Hinge Moment – UI vs API

Core Concept:

The same prompt given to the same model can produce **very different results** depending on whether you use a Chat UI (like ChatGPT) or make a direct API call.

Why the difference occurs:

- Chat UIs are designed for general consumers and add many hidden layers.
- API calls give you raw, direct access to the model.

Key Insight:

This “hinge moment” is the transition point from casual user to professional AI Agent Engineer. Production work and agent development almost always happen through the API.

What Actually Happens in Chat Interfaces

Chat interfaces hide several important processes:

- **Hidden System Prompts:** Invisible instructions that guide the model's behavior and personality.
- **Silent Tool Calling:** The interface may automatically use tools (search, calculators, etc.) without showing you.
- **Different Default Settings:** Optimized parameters (temperature, top_p, etc.) that differ from what you set in API.
- **Post-processing:** Final output is often refined, formatted, or filtered before being shown to the user.

Takeaway:

You have very limited control in Chat UIs, making them unsuitable for building reliable AI agents.

Why Production & Agents Use API

Main Advantages of Using API:

- **Full Control:** You define exact model, parameters, tools, and system instructions.
- **Consistency:** Predictable and repeatable outputs across runs and applications.
- **Cost Management:** Better visibility and control over token usage and spending.
- **Structured Output:** Ability to get clean JSON, tool calls, or other formats ideal for integration into agents and applications.

Conclusion:

For any serious, production-grade, or agentic work, the API is the standard.

Calling LLMs via API

Moving from Chat to Code is a fundamental skill.

Key Components:

- Use the official OpenAI SDK (or equivalent for other providers).
- Configure API key securely (preferably using environment variables).
- Make structured chat completion requests.

Core Idea:

Instead of typing in a web interface, you now programmatically define messages and control every aspect of the generation.

The Three Core Message Roles

1. System Role

- Defines the assistant’s **persona**, expertise, rules, and constraints.
- Acts as the “constitution” of the AI.
- Example: “You are a helpful and concise study guide assistant.”

2. User Role

- Contains the actual request, question, or task from the user/application.
- This is what you want the AI to act upon.

3. Assistant Role

- Represents the AI model’s responses.
- Can be used to continue conversations or **prefill** content.

Best Practice:

Use the System role effectively – it has the strongest influence on the model’s behavior.

Assistant Prefill – Powerful Technique

What it is:

You partially write the beginning of the Assistant’s response in the messages array.

Why it’s powerful:

- Forces the model to follow a specific format or structure.
- Improves consistency, especially for JSON, lists, or step-by-step outputs.
- Reduces unwanted introductory text.

When to use:

When you need strict output formatting (very useful in agent workflows).

Structured Outputs – Must for Agents

Structured outputs are essential when building reliable AI systems.

Three Main Approaches:

1. JSON Mode

- Guarantees valid JSON output.
- Easy to parse in code.
- Simple to implement.

2. Tool Calling (Function Calling)

- Allows the model to dynamically call functions/tools.
- Best for complex, multi-step agent interactions.

3. XML Tagging

- Provides clear hierarchical structure.
- Human-readable and flexible for nested data.

Recommendation:

Always aim for structured output when building agents.

Prompt Anatomy – System + User + Assistant

A complete prompt is usually composed of:

- **System:** Sets context and persona.
- **User:** Gives the specific instruction or task.
- **Assistant (Prefill):** Optionally starts the response to guide formatting.

Combined Effect:

These three parts together form the full input for the API call and determine the quality of the output.

Positive Framing Beats Negative Rules

Golden Rule:

Positive instructions work significantly better than negative ones.

- **Bad (Negative):** “DON’T return JSON, DON’T use Markdown, Do NOT be conversational...”
- **Good (Positive):** “Provide the response as a valid, compact JSON object. Focus only on the core answer, being concise and direct.”

Why it matters:
The model responds better when told what **to do** rather than what **not to do**.

Markdown vs XML Structure

Structure dramatically improves output quality.

- **Markdown:** Simple, clean, and sufficient for most cases (headings, lists, bold, etc.).
- **XML Tagging:** More verbose but excellent for complex hierarchical or nested data.

Best Practice:
Choose one structure and be consistent throughout the prompt.

Zero-Shot vs Few-Shot Prompting

Attribute	Zero-Shot Prompting	Few-Shot Prompting
Examples Provided	None	2–4 high-quality input/output examples
Guidance	Minimal	Strong guidance through examples
Performance	Good for simple tasks	Usually much better, especially for complex or specific tasks
Use Case	Quick tasks	Tasks needing consistent format or reasoning style

Tip:
Start with Zero-Shot, then move to Few-Shot when you need higher consistency.

COSTAR – Your Go-To Framework

COSTAR is a practical and highly effective prompting framework:

- **C – Context:** Provide background, situation, and relevant details.
- **O – Objective:** Clearly define what you want to achieve.
- **S – Style:** Specify desired writing style or format.
- **T – Tone:** Describe the voice and mood (professional, friendly, exciting, etc.).
- **A – Audience:** Define who the output is for.
- **R – Response:** Give exact output format, length, and constraints.

Benefit:
Using **COSTAR** brings structure and clarity, leading to significantly better results.

COSTAR Framework – Detailed Breakdown

Real Example Elements:

- **Context:** Product details, market situation, goals.
- **Objective:** What the output should accomplish.
- **Tone:** Sophisticated yet accessible, innovative, etc.
- **Style:** Feature-benefit storytelling, bullet points, etc.
- **Audience:** Specific user personas.
- **Response:** Word count, format (JSON, Markdown), SEO rules, etc.

Mastering **COSTAR** is one of the fastest ways to improve prompt quality.

The Iterative Refinement Loop

The most important skill in prompt engineering:

1. **Prompt** → Write initial prompt
2. **Test** → Run and observe output
3. **Analyze** → Identify strengths and weaknesses

4. **Improve** → Refine the prompt
5. **Repeat** → Continue until desired quality is reached

Mindset Shift:

Great prompting is rarely one-shot. Professionals iterate.

Common Prompting Mistakes to Avoid

- **Vague Instructions:** Giving overly broad commands (e.g., "Write a story").
 - **Lack of Context:** Not providing necessary background or audience details.
 - **Assuming Persona Knowledge:** Not clearly defining the AI's role.
 - **Ignoring Length/Format Constraints:** Not specifying word count or output structure.
 - **One-and-Done Thinking:** Expecting perfect output on the first try without iteration.
-

Session 1 – Key Takeaways

1. **Define the Persona** — Always clearly state the role and expertise.
2. **Establish Clear Objectives** — Be specific about what you want.
3. **Provide Critical Context** — Give all relevant background.
4. **Use Structured Frameworks** — COSTAR is highly recommended.
5. **Embrace Iteration** — Treat prompting as a refinement process.
6. **Set Explicit Constraints** — Define format, length, tone, and boundaries.

Final Advice:

Master the fundamentals in this session before moving to advanced reasoning techniques in Session 2.
